

O Ataque do Gavião (Ave de Rapina 3D)

Guia Didático: Equação da Parábola, Física de Mergulho, Inclinação Dinâmica (Pitch), Sombra com Altitude e Colisão Esférica

1. Visão Geral: O Predador que Desce dos Céus

Enquanto a Raposa é uma ameaça terrestre que patrulha a grama em velocidade constante, o **Gavião (Ave de Rapina)** funciona como uma ameaça aérea dinâmica. Ele surge a grandes altitudes fora do campo, executa um mergulho rasante veloz e curvado mirando na posição onde o coelho estava, e imediatamente retorna às nuvens pelo outro lado da arena.

Para testar o mergulho a qualquer momento durante o jogo, o usuário pode simplesmente pressionar a tecla 'A'.

2. Dicionário de Variáveis do Gavião

As seguintes variáveis em `AveRapina3D.hpp` e `AveRapina3D.cpp` controlam todo o estado do predador aéreo:

Variável	Tipo	Valor Padrão	Função no Jogo
<code>aveActive</code>	<code>bool</code>	<code>false</code>	Indica se o gavião está ativo e executando a passagem de ataque.
<code>aveX</code> , <code>aveY</code> , <code>aveZ</code>	<code>float</code>	Início: <code>(14.0, 7.5, alvoZ)</code>	Coordenadas tridimensionais em tempo real. X é o deslocamento, Y é a altitude e Z é a profundidade.
<code>aveAlvoX</code> , <code>aveAlvoZ</code>	<code>float</code>	Coelho no spawn	Ponto registrado no chão onde o coelho estava quando o gavião iniciou o mergulho.
<code>aveVelocidadeX</code>	<code>const float</code>	<code>0.16f</code>	Passo de deslocamento horizontal para a esquerda (sentido <code>-X</code>) a cada frame.
<code>avePhase</code>	<code>float</code>	Incremento <code>0.35 rad</code>	Fase angular periódica da animação do bater sincronizado das asas.
<code>aveJaTirouVida</code>	<code>bool</code>	<code>false</code>	Trava de segurança: garante que a ave desconte no máximo 1 vida por mergulho.
<code>framesAteProximaAve</code>	<code>int</code>	12 a 22 segundos	Temporizador aleatório para agendar os ataques aéreos surpresa.

3. A Física do Mergulho: A Equação da Parábola 3D

No espaço 3D, a trajetória de mergulho e escape do gavião é modelada por uma **função quadrática (parábola com vértice no ponto mais baixo)** no plano XY, fixando a profundidade Z no corredor de ataque:

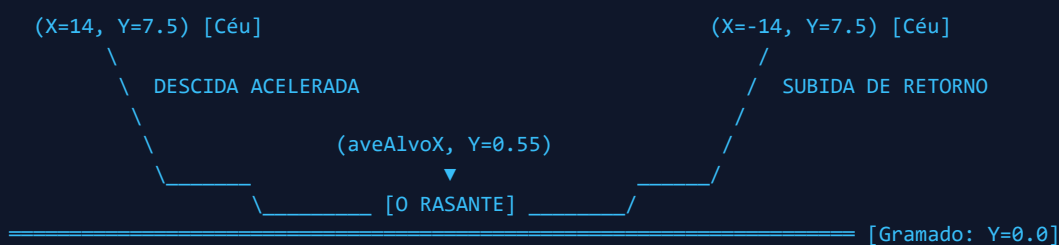
Equação da Trajetória:

$$Y(X) = a \times (X - \text{aveAlvoX})^2 + 0.55$$

Onde os termos significam:

- **aveAlvoX:** A coordenada horizontal do vértice da parábola (onde o rasante acontece).
- **K = 0.55f:** A altitude mínima da ave no rasante (exatamente a altura do corpo do coelho!).
- **Coeficiente de Abertura (a):** Calculado dinamicamente para que a ave sempre comece no céu em $Y = 7.5$ no ponto de entrada $X = 14.0$:

$$a = (7.5 - 0.55) / (14.0 - \text{aveAlvoX})^2$$



4. Dinâmica de Rotação (Pitch): O Bico que Mira na Presa

Se a ave apenas seguisse a parábola com a inclinação fixa, pareceria um objeto rígido e estático flutuando no ar. Para criar realismo orgânico, o código calcula a **derivada instantânea da parábola** a cada frame no `main.cpp`:

```
// Derivada de Y em relacao a X: dY/dX = 2 * a * (X - aveAlvoX)
float dx = aveX - aveAlvoX;
float inclinacaoGraus = std::atan(2.0f * a * dx) * 180.0f / PI_F;

// Aplica a rotacao em torno do eixo Z:
glRotatef(-inclinacaoGraus, 0.0f, 0.0f, 1.0f);
```

🔗 Como essa rotação se comporta durante o voo?

1. **Na Descida ($\text{aveX} > \text{aveAlvoX}$):** O valor da inclinação é positivo, fazendo o bico e as garras apontarem agressivamente para baixo, em ângulo de ataque direto na presa.
2. **No Rasante ($\text{aveX} == \text{aveAlvoX}$):** A derivada é zero! O corpo da ave se nivela horizontalmente com o gramado.
3. **Na Subida ($\text{aveX} < \text{aveAlvoX}$):** A derivada inverte o sinal, fazendo o bico apontar para o céu enquanto o gavião ganha altitude e foge.

5. Animação Orgânica: O Bater de Asas 3D

Ao contrário do jogo 2D onde as asas eram triângulos planos, no 3D as asas possuem envergadura física aberta ao longo do eixo Z.

- A fase do bater de asas avança continuamente: `avePhase += 0.35f;`
- O ângulo instantâneo da asa é gerado por uma onda senoidal:

$$\text{wingAngle} = \sin(\text{avePhase}) \times 38.0^\circ$$

- As asas são articuladas em torno do eixo longitudinal X:
 - **Asa Esquerda (+Z):** `glRotatef(wingAngle, 1.0, 0.0, 0.0);`
 - **Asa Direita (-Z):** `glRotatef(-wingAngle, 1.0, 0.0, 0.0);`

O sinal oposto (`+wingAngle` e `-wingAngle`) garante que ambas as asas subam e desçam juntas e simetricamente em relação ao dorso do gavião.

6. Pista Visual de Perigo: A Sombra Dinâmica no Solo

Em um ambiente 3D sob projeção de perspectiva, quando um predador desce das alturas, o jogador tem dificuldade de saber sua posição exata no chão apenas olhando para cima. A função `drawSombraAve()` resolve isso projetando um disco escuro no gramado em `(aveX, aveZ)` que reage à altitude:

💡 Indicador Visual de Proximidade (Feedback ao Jogador)

- **Gavião Alto no Céu ($Y = 7.5$):** A sombra fica **grande** (raio = 0.70) e **muito clara/difusa** (alfa = 0.12). O jogador percebe que há algo sobrevoando.
- **Gavião Próximo ao Chão ($Y = 0.55$):** A sombra **encolhe** (raio = 0.35) e fica **escura e nítida** (alfa = 0.40). Isso funciona como uma mira laser avisando: "*Saia daqui imediatamente!*".

7. O Sistema de Colisão e a Mecânica de Esquiva

No arquivo `SistemaColisao3D.cpp`, a função `verificarColisaoComAve()` roda a cada frame e calcula a distância euclidiana tridimensional entre o gavião e o coelho:

$$\text{distancia} = \sqrt{(\text{aveX} - \text{coelhoX})^2 + (\text{aveY} - \text{coelhoY})^2 + (\text{aveZ} - \text{coelhoZ})^2}$$

```
void verificarColisaoComAve() {
    if (!aveActive || aveJaTirouVida || coelhoEscondido) return;

    float dx = aveX - coelhoX;
    float dy = aveY - coelhoY;
    float dz = aveZ - coelhoZ;
    float distancia = std::sqrt(dx * dx + dy * dy + dz * dz);

    float raioColisao = 0.95f;

    if (distancia < raioColisao) {
        if (rabbitLives > 0) rabbitLives--;
        aveJaTirouVida = true; // Trava: tira no maximo uma vida por passagem
    }
}
```

⚠ Como o Jogador se Esquiva com Sucesso?

O gavião não possui mira teleguiada em tempo real. No momento em que surge (`spawnAve`), ele memoriza (`aveAlvoX`, `aveAlvoZ`).

Isso cria a oportunidade de gameplay:

- Se o jogador ficar **parado**, ele certamente será atingido no rasante em `Y = 0.55`.
- Se o jogador **correr para os lados** (alterando seu Z) ou **avançar/recuar** (alterando seu X), ele sai do cone de impacto de 0.95 unidades e o gavião mergulhará atingindo apenas o ar vazio!
- Se o jogador estiver na **Câmera em 1ª Pessoa**, ele pode usar as teclas **W / S** para inclinar a visão para cima e assistir ao gavião mergulhando em sua direção antes de se esquivar!

8. Ciclo Completo de Vida do Ataque

Fase	Posição X	Comportamento Físico e Gráfico
1. Spawn	<code>X = 14.0</code>	Trava as coordenadas do coelho (<code>aveAlvoX</code> , <code>aveAlvoZ</code>). Começa em <code>Y = 7.5</code> .
2. Mergulho	<code>14.0 > X > alvoX</code>	Desce rapidamente seguindo a parábola. O bico inclina para baixo. Sombra encolhe e escurece.
3. O Rasante	<code>X == alvoX</code>	Ponto mais baixo da trajetória (<code>Y = 0.55</code>). Checa colisão com o coelho (<code>< 0.95</code>).
4. Subida	<code>alvoX > X > -14.0</code>	Ganha altitude em direção ao céu. Bico inclina para cima. Sombra clareia e se expande.
5. Despawn	<code>X < -14.0</code>	Sai do campo pelo lado esquerdo. <code>aveActive = false</code> . Sorteia novo ataque em 12 a 22s.